Bolt.new: When AI Builds the Builder

Teaching Case (HBS-style) — written for this course

Prepared for: TUM Entrepreneurship Course: Prototyping Entrepreneurial Ideas for AI

Decision Point: July 2024 — $700K ARR After Seven Years, a Shelved AI Prototype, and 90 Days to Ship or Shut Down

How this case ties into Module 06

This module's core question — what is the fastest thing you can build to learn? — is Bolt's whole biography. Work three connections as you read: (1) **Speed as strategy:** Bolt.new shipped in roughly 90 days and iterated in public; this is the Weekend-Build ethos at company scale, with payroll and a board deadline attached. (2) **Build to learn (Idea 4):** StackBlitz ran no survey, no focus group, no landing-page test — the prototype WAS the product test, and the market graded it in revenue. (3) **The workshop bridge:** in today's Show & Build you demo your own 48-hour builds and take brutal feedback. Bolt is the pep talk with revenue numbers attached: same playbook, more zeros.

Introduction

In June 2024, Eric Simons and the team at StackBlitz faced an unusual technological moment: the tools for building software had fundamentally changed overnight. Anthropic had just released Claude 3.5 Sonnet, and for the first time a large language model could reliably write production-quality code from text descriptions. The constraint that had defined software development for 50 years—that you needed to know how to code—was evaporating.

StackBlitz, founded in 2017 by Eric Simons and Albert Pai, had built WebContainers: a browser-based Node.js runtime that let developers run full-stack applications directly in the browser, no servers required. Announced in 2021 after four years of engineering, WebContainers had remained a niche tool for education and quick prototyping. Millions of developers used it; almost nobody paid for it. Seven years in, StackBlitz had stalled at roughly $700K in annual recurring revenue and was—by the founder's own telling—weeks

from shutting down. But in 2024, the technology suddenly became the perfect foundation for something much bigger: an AI that could generate entire web applications from a prompt.

Simons had already watched the AI wave disrupt software development. Cursor and GitHub Copilot had turned coding into something more like dictation. Vercel (the dominant deployment platform for web apps) had launched v0, an AI component generator. Replit (a competitor) was racing toward an autonomous coding agent. Dozens of "AI website builders" were launching every month. The market was flooded with founders trying to ride the LLM wave.

The question was not whether AI-generated code was possible. The question was: **How do you build an AI app generator that is genuinely useful, when everyone and their cousin is building one?**

Simons knew he had one real advantage: WebContainers meant code could run instantly, in the browser, without waiting for servers. He even had a head start: he had prototyped an AI app builder in February 2024, then shelved it because the models weren't good enough. Claude 3.5 Sonnet changed that. But he also knew the clock was moving fast—and this time it was existential. If he couldn't ship something in the roughly 90 days before his next board meeting that was obviously better than Cursor, v0, and Replit, there might not be a StackBlitz left to be yesterday's news.

Background

The Evolution of Development Tools (2017-2024)

To understand StackBlitz's position in 2024, you need to understand the trajectory of web development tools:

2017-2019: The Age of Local Development - Developers wrote code locally (on their laptop) - Pushed to Git (GitHub, GitLab) - Deployed to servers (AWS, Heroku, Vercel) - Development loop: write → save → refresh → debug - Friction: environment setup, dependency hell, slow iteration

2019-2021: The Rise of Cloud IDEs - GitHub Codespaces, Replit, StackBlitz, Gitpod emerged - "Code in the browser" became feasible - Reduced friction for beginners (no local setup) - Still had latency: browser talks to server, server runs code

StackBlitz's WebContainers (2021-2024) - WebContainers (announced in public beta at Google I/O in May 2021, after four years of engineering) was a breakthrough: Node.js

running directly in the browser - Meaning: full-stack code could execute instantly, in-browser, no server round-trip - Development loop: write → save → result appears instantly - Uniqueness: No other platform could do this at scale

The LLM Wave Hits (2022-2024) - GPT-3 (2020) could write code, but quality was inconsistent - GitHub Copilot (2021) brought LLM-powered code completion to IDEs (VS Code) - ChatGPT (November 2022) made LLMs accessible to non-technical users - GPT-4 (March 2023) and Claude 3 (early 2024) wrote increasingly solid code; Claude 3.5 Sonnet (June 2024) was the step change—agentic coding good enough to build whole apps - Realization: The bottleneck in software development was no longer technical execution. It was intent translation (telling the computer what you wanted).

StackBlitz Before Bolt (2017-2024)

StackBlitz had built beloved technology and a stalled business: - Platform: browser-based IDE with instant web preview - Use cases: education, quick prototyping, open-source demos, code snippets - Users: developers, teachers, learners, open-source maintainers — millions of monthly users of StackBlitz-powered environments (company-reported), concentrated among educators and open-source developers - Revenue: Freemium model (roughly $10-30/month premium) - Annual recurring revenue: stalled at ~$0.7M after seven years — not profitable, and burning through a $7.9M seed round raised in April 2022 - Team: roughly 20 people

The platform was not “cool” in startup terms. It was useful and widely used, but revenue was flat. Most professional developers still preferred local IDEs (VS Code) and were skeptical of “browser IDEs” being good enough.

But StackBlitz had something none of the new entrants had: a seven-year head start on building a browser runtime that actually worked at scale.

The Competition Emerges (2023-2024)

By mid-2024, the “AI app builder” space was getting crowded:

Competitor	Founded	Funding	Focus	Key Feature
GitHub Copilot	2021	GitHub/Microsoft	Code completion in IDEs	IntelliSense-style assistance
Cursor	2022	Series A ($60M, Aug 2024)	AI-native IDE	Chat interface for coding
Replit	2016	Series B ($97M+, 2023)	Browser IDE + AI	Multi-language; agent in the works
Vercel v0	2023	Vercel internal	Component generation	React component builder
Durable	2021	Series A	Website builder + AI	Drag-and-drop + LLM
Webflow AI	2024	Webflow internal	Design + code for websites	Design tool integration
10web	2017	Series A	WordPress site builder + AI	WordPress-focused
StackBlitz Bolt	2024	TBD	Full-stack app generator	Browser runtime advantage

The Crowding Problem: 20+ AI app builders, website builders, and code generation tools were competing for developers' attention. Differentiation was difficult. Most were variations on the same theme: "Write your code goal in English, get a working app."

The Technological Moment in Mid-2024

By mid-2024, several technological changes collided:

1. **LLM Maturity**: Claude 3.5 Sonnet, GPT-4o, and others could write entire applications (frontend + backend) in a single prompt. With the June 2024 Sonnet release, code quality crossed from scaffolding to production-ready.
2. **Browser AI Integration**: AI models were becoming smaller, faster, cheaper. Running model inference in the browser (using WebGPU or API access) was practical.
3. **Instant Feedback Loops**: WebContainers meant full-stack code could run instantly. A user could describe an idea, the LLM would generate code, and they'd see the result in <5 seconds.

4. **Accessibility Explosion**: Non-programmers could now create functional apps without ever writing code. This was novel and powerful.

The combination suggested something: an AI app generator that was live, instant, and interactive could be genuinely useful—not just a novelty.

The Situation

It's July 1, 2024. Eric Simons is in a meeting with his leadership team at StackBlitz. The discussion is urgent—and existential:

The Opportunity: - LLMs can now generate full-stack applications - WebContainers let you run them instantly, in-browser - This combination is uniquely powerful and difficult for competitors to replicate - The market for "AI-powered development" is exploding - First-mover advantage matters—the "default" AI builder will win mindshare

The Pressure: - Vercel v0 already exists and has traction (generated by Vercel team, integrated with their platform) - Replit, Cursor, and others are adding AI app generation features - If StackBlitz doesn't move fast, it'll be seen as "another player" instead of a leader - Investor interest in AI companies is at peak. VCs want portfolio companies to have "AI strategy." - The company itself is running out of road: ~$700K ARR after seven years, the April 2022 seed nearly spent, and a board meeting in roughly 90 days

The Question: Should StackBlitz bet what's left of the company on Bolt.new, an AI app generator powered by WebContainers?

The Decision Landscape

Option 1: Move Fast, Ship in 2 Weeks (MVP Approach)

Build a minimal, viable AI app generator: - User enters a prompt: "Build a todo app with React" - Backend sends prompt to Claude API - Claude generates code (frontend + backend) - Code runs instantly in WebContainers - User can edit code or request changes - Ship to a waiting list within 2 weeks

Pros: - First-mover advantage; you get out before competitors iterate - Validate core idea quickly: Can users adopt this? Does it work? - Market momentum: "StackBlitz just launched Bolt" becomes news - Team morale: shipping fast is exciting

Cons: - Rough around the edges (code generation might be buggy, UX unpolished) - Competitors launch their versions 2 weeks later, learn from your mistakes - Technical debt:

rush job might collapse under load - Quality expectations: if it fails for early users, reputation damage

Option 2: Polish for 2 Months, Ship When Ready

Build a more complete product: - Robust error handling (LLM generations sometimes fail; you need graceful fallbacks) - Better UX (iteration UI, edit mode, preview toggles) - Backend optimization (serve code generation from optimized models, cache common patterns) - Extensive testing (edge cases, different prompt types) - Ship to general availability

Pros: - Higher quality launch; users are delighted, not frustrated - Your version is meaningfully better than early versions of competitor solutions - Technical foundation is solid; can scale with demand - You look professional, not rushed

Cons: - Competitors ship faster and win "first mover" narrative - You risk missing the hype window (AI app builders are a trend; trends move fast) - Two months is an eternity in AI. Models improve, new competitors emerge, market moves on - Your team sits in "building mode" for 8 weeks; no other projects ship

Option 3: Hybrid Approach (Tiered Launch)

Release incrementally: - **Week 1-2**: Limited beta launch (100 users from waitlist). Validate core functionality. - **Week 3-4**: Expand beta, gather feedback, fix critical issues - **Week 5-8**: Broader launch with improvements based on feedback - **Week 8+**: Full public launch

Pros: - Get market signal early; adjust course based on feedback - Manage risk: catch bugs before they hit millions of users - Still perceived as "first mover" (early beta counts as launching early) - Can claim "from the makers of WebContainers" right away

Cons: - More complexity to manage (multiple release phases, communication) - Delays full public launch (competitors might beat you to the "real" launch) - Beta users might complain publicly if experience is rough

The Differentiation Question

Even if StackBlitz ships Bolt quickly, what makes it different from v0, Replit, Cursor, etc.?

Potential Advantages: 1. **WebContainers**: Only StackBlitz has native, in-browser Node.js runtime. Code runs instantly, no server latency. 2. **Full-stack capability**: Generate both frontend and backend code that runs in-browser. 3. **Developer-first UX**: Unlike Webflow or Durable, Bolt targets developers (and advanced users), not designers. 4. **Open**: Unlike v0 (Vercel-specific), Bolt works with any framework, any stack.

Challenges: 1. **WebContainers is niche**: Most developers don't know about it. Advantage is real but not obvious. 2. **Code quality matters**: If Claude/GPT-4 generates the code, all AI

builders will have similar quality. The LLM is the bottleneck, not StackBlitz's infrastructure. 3. **Requires continuous improvement**: As LLMs improve, Bolt needs to improve. Bug fixes, edge cases, user feedback loops. 4. **Market is crowded**: Even with advantages, you're competing on execution and polish, not defensibility.

The Scope Question: What's the Cheapest Thing That Answers the Question?

This is crucial. "AI app generator" is broad. The discipline isn't "what's the minimum viable product" in the abstract—it's: what is the cheapest build that answers the question will people use this and pay for it?

Narrow Scope Option 1: React-only - Only generate React apps - Only frontend + basic Node.js backend - Target: React developers who want to prototype quickly

Narrow Scope Option 2: Full-stack, but specific use cases - Focus on specific templates: todo apps, dashboards, landing pages, blogs - Pre-baked patterns; Claude fills in the details - Higher quality for narrow scope

Narrow Scope Option 3: Interactive code generation - Less focus on "one prompt, full app" - More focus on "iterate with AI": describe feature, AI adds it, preview it, iterate - Closer to a "pair programmer" than a builder

Narrow Scope Option 4: API-first - Don't build consumer-facing web interface first - Offer Bolt as an API: developers can build their own UI on top - Simpler to execute, but less consumer mindshare

The scope choice determines everything downstream: positioning, marketing, user base, roadmap.

Resource and Timeline Constraints

StackBlitz has: - ~15-20 person team (Simons cut seven or eight roles in July to stretch the runway) - ~$0.7M annual recurring revenue (stalled; burning cash) - Strong engineering; weak in marketing/sales - A $7.9M seed round (April 2022), now nearly spent — runway measured in months, not years - WebContainers technology is proven, stable

For a 2-week MVP: - 5-8 engineers full-time - 1-2 product managers - Minimal design work (use existing StackBlitz UI) - No marketing prep

For an 8-week launch: - Same team, but can do more quality work - Time for design iteration - Time for documentation - Time for marketing/PR prep

For both timelines: **You're going all-in.** The team working on Bolt is the team; there is no second bet if this one misses.

The Exhibits

Exhibit A: The "AI App Builder" Landscape (Summer 2024)

Illustrative reconstruction for classroom use — directionally faithful, individual ratings are teaching estimates.

Product	Approach	Time to First Result	Code Quality	Iteration	Developer Adoption
Cursor	AI IDE (code completion)	Instant (autocomplete)	High (human-assisted)	Excellent	High (developers love it)
GitHub Copilot	Code completion in VS Code	Instant (suggestion)	High	Excellent	Very High
Vercel v0	Component generator	10-30 seconds	Medium-High	Good (edit code, regenerate)	Medium (Vercel ecosystem)
Replit Agent (shipped that September)	Full-stack generator	20-60 seconds	Medium (cloud-based)	Okay	Low-Medium
Durable	Website builder + AI	1-2 minutes	Medium (design-first)	Poor (not code-centric)	Low (non-developers)
Bolt.new (planned)	Full-stack in-browser	<5 seconds (runs instantly)	High (Claude 3.5 Sonnet)	Excellent (live editing)	? (TBD)

Key insight: Speed of feedback loop is a differentiator. Bolt.new's instant execution is its edge.

Exhibit B: User Segmentation and Potential Market Size

Illustrative reconstruction for classroom use — directionally faithful, individual numbers are teaching estimates.

Segment	Size	Use Case	Willingness to Adopt Bolt
Professional developers	~25M worldwide	"Quick prototype, but I'll polish in my IDE"	Medium (might use for rapid mockups)
Developer-learners	~5M (in boot camps, self-teaching)	"I want to learn by doing, not typing"	High (instant feedback is motivating)
Non-programmer creators	~500M (marketers, designers, founders)	"I have ideas but can't code"	Very High (enables them to build)
Educators	~500K relevant	"Teach web dev without setup friction"	Very High (StackBlitz is already big here)
Enterprise	~10K companies	"Rapid prototyping, internal tools, demos"	Medium (requires reliability, support)

Market size estimate: - If Bolt captures 1% of professional developers (250K users) at $10/month = $30M ARR - If Bolt captures 5% of learners (250K users) at $5/month = $15M ARR - If Bolt captures 0.1% of non-programmers (500K users) at $5/month = $30M ARR - **TAM estimate: $75M+ (if successful)**

Exhibit C: Timeline and Risk Analysis

Illustrative reconstruction for classroom use — directionally faithful, individual numbers are teaching estimates.

2-Week MVP Launch: | Week | Deliverable | Risk | Contingency | |—|—|—|—| | 1 | Claude integration + basic code gen | API failures, rate limits | Fallback to GPT-4 | | 2 | UI polish, deployment, waitlist signup | Capacity issues, bugs in gen code | Limited beta (100 users) | | 2 (public) | Announce, generate buzz, capture demand | Overwhelmed servers, poor user experience | Scale back beta, extend timeline |

Risks: - Code generation failures (LLM produces non-working code) → User frustration - Capacity/scaling issues → Service goes down, reputation hit - Integration bugs → Claude API misuse, account bans - UX gaps → Users confused about how to use Bolt - Competitive response → v0/Replit launch their own versions immediately

8-Week Polish Launch: | Phase | Deliverable | Risk | |—|—|—| | Weeks 1-3 | Prototype + user testing | Wrong direction, poor feedback | | Weeks 4-5 | UX refinement, edge case handling | Over-polishing, diminishing returns | | Weeks 6-7 | Scale testing, documentation | Bugs discovered late, rushed fixes | | Week 8 | Launch | Market has moved on |

Risks: - Market moves faster than 8 weeks; trends shift - Competitors launch better versions while you polish - Opportunity cost: team isn't working on other products

Exhibit D: The Competitive Response Prediction

Illustrative reconstruction for classroom use — a forecast exercise, not reported events.

If StackBlitz ships Bolt quickly:

Week	Vercel Response	Replit Response	Microsoft/OpenAI Response
Week 1-2	Observes launch	Observes launch	Observes launch
Week 3-4	Improves v0, emphasizes "Vercel integration"	Pushes "Replit Agent" improvements	ChatGPT gets app-building features
Week 5-8	v0 gets better iteration UI, mobile support	Replit Agent can edit existing apps	Claude/OpenAI release their own builders
Month 3+	v0 becomes the default for Vercel users	Replit dominates multiplayer collaborative building	OpenAI partnership ecosystem beats everyone

Implication: StackBlitz's first-mover advantage lasts maybe 2-4 weeks before competitors catch up. After that, it's about execution and user love, not timing.

Exhibit E: The Hype Cycle Reality Check

Illustrative reconstruction for classroom use — dates verified, "hype" and "status" columns are teaching judgments.

Looking at similar launches in 2022-2024:

Product	Hype Peak	Reality Check	Status (Mid-2024)
Stable Diffusion (Aug 2022)	Enormous (changed AI forever)	Works great for some uses, terrible for others	Mature, lots of competition, consolidating
ChatGPT (Nov 2022)	Enormous (over 1M users in days)	Reliability, context limits, costs high	Dominant; GPT-4o just shipped
Midjourney (July 2022)	Very high (artists excited, scared)	Quality improved, but not replacing artists	Successful, profitable, growing
ChatGPT Code Interpreter (July 2023)	Moderate (excited developers)	Actually useful for quick tasks	Rapidly improving, changing market

Lesson: Hype peaks within 2-4 weeks of launch. First-mover gets the peak. But sustained success requires delivering on the hype.

The Decision

It's July 1, 2024. You are Eric Simons, CEO of StackBlitz. Your next board meeting is roughly 90 days out, and your runway is measured in months: are you building Bolt.new? If yes, what's your launch date and scope?

Immediate Decisions You Face

Decision 1: Do You Build Bolt at All?

Yes/No question. Reasons to say no: - The company is fragile. A failed bet burns the runway you have left. - The market is too crowded. Odds of winning are low. - Your team is focused on StackBlitz platform. Splits resources.

Reasons to say yes: - WebContainers is a real advantage that no one else has. - Market timing is now. Waiting means you miss the wave. - Your user base (developers, educators) naturally wants this. - First-mover advantage in narrative: "StackBlitz launches AI app builder" is news.

Decision 2: Scope—What Do You Actually Build?

If you ship fast, your first version is: - User prompt → LLM generates code → code runs in WebContainers → user sees result - Can user edit the generated code? (Yes, probably—this

is a major feature) - Can user request changes, and AI regenerates? (Yes—essential for iteration) - What frameworks? (React? Vue? Svelte? All three?) - Does backend code run too? (Yes—this is the WebContainers advantage) - Can users save their creation? (Yes, but maybe only authenticated users) - Can users share? (Yes, via public URLs—huge for virality)

Decision 3: What's Your Differentiation Story?

You need a narrative that's not "we're another AI builder": - Option A: "Full-stack code generation. Backend runs in-browser. No server latency." - Option B: "Instant feedback loops. See your idea come to life in 3 seconds." - Option C: "The AI copilot for rapid prototyping. Built for developers who want to move fast." - Option D: "Build without servers. Generate, edit, run, ship—all in one place."

Pick one. This is your positioning for the next year.

Decision 4: Launch Timeline—When Do You Ship?

- Option A: 2 weeks (high-risk, high-reward)
- Option B: 4 weeks (balanced approach)
- Option C: 8+ weeks, using the full runway to the board meeting (safer, higher quality)

Each timeline affects user expectations, competitive response, and team morale.

Decision 5: Who's the Initial User?

You can't serve everyone. Pick your wedge: - **Professional developers** (builders, rapid mockups, internal tools) - **Learners** (educational context, low stakes, learning-focused) - **Non-programmers** (democratization angle, "anyone can build") - **Educators** (StackBlitz is already strong here)

Initial user determines your marketing message, feature priorities, and success metrics.

The Deeper Question

What does StackBlitz want to be in 2025?

- A developer tools company (invest in Bolt, reposition StackBlitz as "platform")
- A niche education platform (focus on teachers and learners)
- A browser IDE company (stay focused on core product)
- An AI-first company (everything you build has AI baked in)

This decision cascades to everything else: team, funding, partnerships, competitors, narrative.

Study Questions for Class Discussion

1. **If you ship Bolt in 2 weeks and it's rough, is that a feature or a bug?** How does the market respond to "shipped fast but buggy" vs. "took 8 weeks and polished"? What's the cheapest version that still answers the question you actually need answered—"will anyone use this and pay?"
2. **WebContainers is StackBlitz's unfair advantage. But how defensible is it really?** Could Vercel or Microsoft replicate this within 6 months if they wanted to?
3. **Is Bolt a new product or a feature of StackBlitz?** Does branding it separately help (separate identity, clear positioning) or hurt (spreads resources, confuses market)?
4. **Who actually wins in the AI app builder market?** Is it the fastest shipper, the best product, the best integration, or the one with the strongest distribution? What does this imply for StackBlitz?
5. **If you're Simons and you choose to build Bolt, what's your kill decision criteria?** When do you admit it's not working and shut it down—especially when the same 90 days decide whether the company itself survives?
6. **The LLM is the core technology. But StackBlitz doesn't own Claude or GPT-4. Does that matter?** What if OpenAI decides to compete with Bolt directly via ChatGPT interface?
7. **What's the sustainable competitive advantage, if any, in this market?** Is there a world where StackBlitz's Bolt doesn't eventually get out-gunned by Microsoft (GitHub Copilot), Vercel (v0), or OpenAI?

Epilogue (read after class)

Spoiler warning: this section gives away what StackBlitz actually did. Read it after the class discussion, not before.

Simons went all-in. In early July 2024 he cut seven or eight roles to stretch the runway, pointed the remaining ~15-person team at the February prototype, and set one deadline: ship before the next board meeting, roughly 90 days out. Bolt.new launched on **October 3, 2024** — no press release, no waitlist theater, a single tweet.

What happened next made it one of the fastest-growing software products on record:

- Built on Claude 3.5 Sonnet (via Anthropic's API) plus WebContainers: the LLM writes the code, the browser runs it instantly

- **$4M ARR within four weeks** of launch (company-reported)
- **~$20M ARR roughly two months after launch**, per the company's own claims; independent press places the $20M crossing at two to three months — either way, a revenue ramp almost nobody had seen before
- **~$40M ARR and 3M+ registered users by March 2025**, five months in, with a team still under 40 people
- Funding followed revenue, not the reverse: a $22M Series A in November 2024, then a $105.5M Series B in January 2025 led by Emergence Capital and GV at a ~$700M valuation

The competitive predictions in Exhibit D mostly came true. Lovable launched about seven weeks after Bolt and posted a comparably steep curve; Replit Agent, v0, and Cursor all kept growing. The first-mover narrative lasted weeks, exactly as the case anticipated. What endured was the asset that was seven years old, not seven weeks old: WebContainers made Bolt's feedback loop feel instant while competitors waited on cloud VMs. And note Exhibit B's cautious "TAM estimate" of $75M — reality blew past half of it in five months, a lesson in how badly we size brand-new categories.

Be precise about what this story is and isn't. It is not "build it and they will come": StackBlitz built the technology in 2017, and for seven years almost nobody paid. It is a company that kept a deep technical asset alive until model quality crossed a threshold, then shipped the cheapest thing that could answer the question will anyone use this and pay? — and let the public answer in revenue. The prototype WAS the product test. Two weeks of live usage settled arguments that two years of internal debate never could. The LLM itself was a commodity everyone could rent; the variance was in the feedback loop wrapped around it, and in picking a wedge (developers who want to prototype fast) instead of chasing everyone.

The case illustrates that speed is not recklessness: Bolt shipped in 90 days on top of seven years of infrastructure. The fastest thing you can build to learn is only that fast because of what you already know how to build.

Teaching case written for this course from public sources. Exhibits marked as illustrative are reconstructions for classroom use.